Stream Processing

answers while the events are still flowing

Watch the 4-min video

In one line: a stream processor reads an **unbounded** flow of events (usually from Kafka) and updates results continuously; the hard parts are **time** (event time vs processing time), **windows** for aggregation, **watermarks** that decide when a window is complete, and **state** that survives crashes through checkpoints.

1 Answers in seconds

A stolen card is used five times in two minutes in three countries. A nightly batch job (lesson 31) would notice tomorrow, long after the money has gone. Fraud scoring, alerting, live dashboards and real-time features need results within seconds while events keep arriving, never "complete". Frameworks such as **Apache Flink**, **Kafka Streams** and **Spark Structured Streaming** (micro-batches by default) handle parallelism, state, failure recovery and time. Kafka keeps the input durably, so a job can always replay from an earlier offset.

2 Event time vs processing time

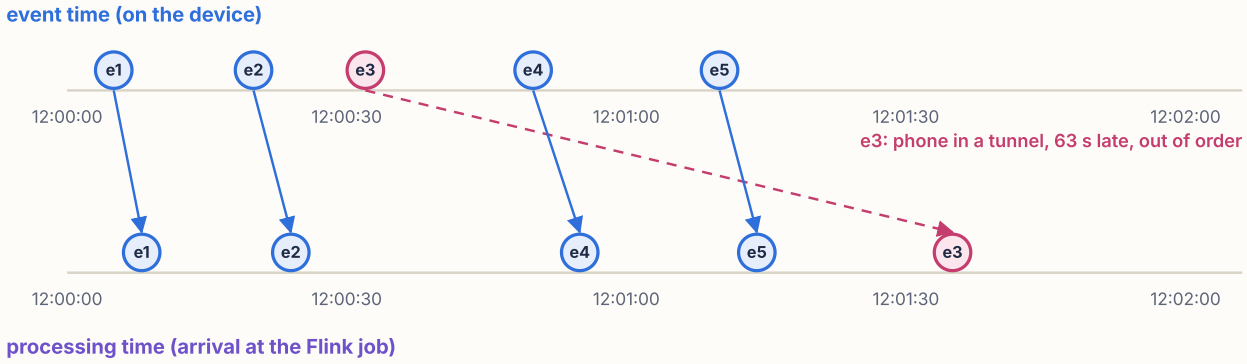


Figure 1. Events usually arrive a little after they happen, but sometimes much later and out of order.

Event time is when something happened, stamped by the source. **Processing time** is when your job sees it. Networks, retries, batching and offline phones make the gap variable. Counting by processing time is easy but wrong: a network hiccup moves events into the wrong minute, and replaying yesterday's data would put it all into today. Counting by event time gives the same answer however late or fast the data arrives, which is what makes replays correct (lesson 33).

3 Windows

An endless stream has no "total"; aggregations need boundaries. A **window** groups events by event time so each group can be computed and emitted. Choosing the window is a product decision: what question is the number supposed to answer?

A sliding window of size 5 min and slide 1 min puts each event in 5 windows, so cost grows with size ÷ slide. For "the last N minutes, right now" on every event (fraud), many systems skip windows and keep a per-key buffer of recent events in state, evicting old ones.

Window	Definition	Example question
Tumbling	fixed size, no overlap; each event in exactly one window	orders per minute
Sliding (hopping)	fixed size, advances by a smaller slide; windows overlap	error rate over the last 5 min, every minute
Session	activity separated by a gap of inactivity; size varies per key	length of one user's visit
Global + trigger	no time bounds; you decide when to emit	every 100 events per key

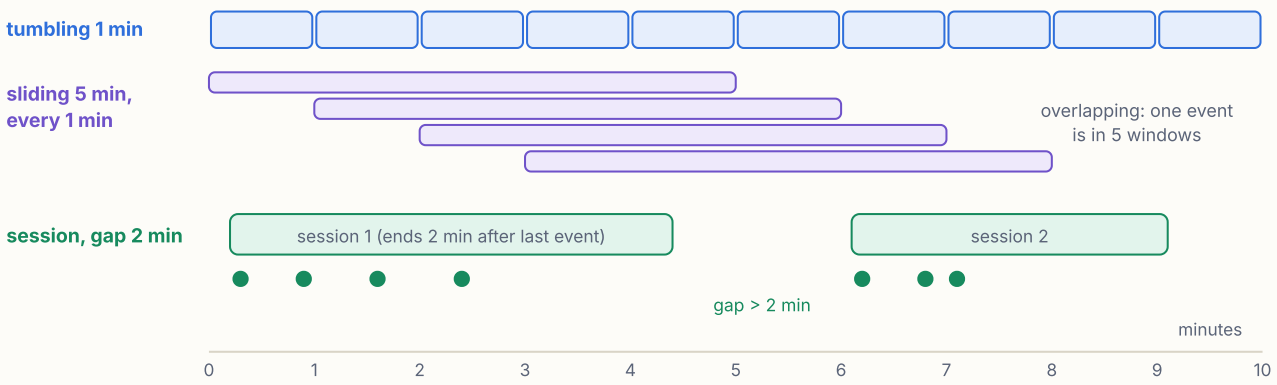
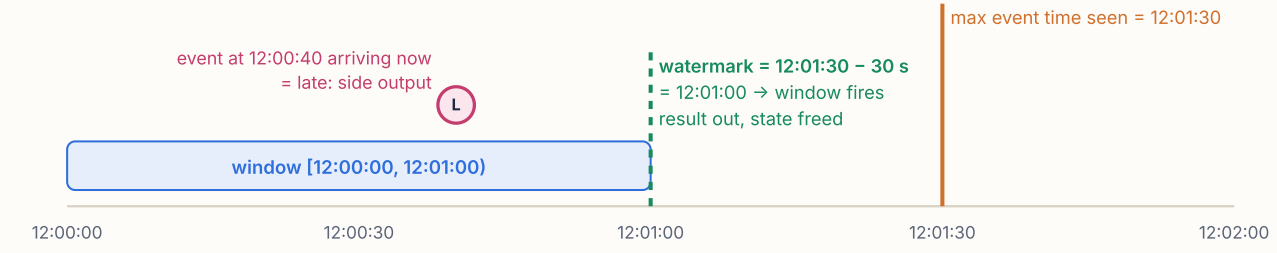


Figure 2. Three window types over the same timeline. The question being asked decides which one you need.

4 Watermarks: when is a window complete?

If events can be late, when can the job emit the result for 12:00–12:01? A **watermark** is the processor's assertion that no more events with an event time earlier than *t* are expected. The common strategy is **bounded out-of-orderness**: watermark = highest event time seen − allowed delay (in Flink, `WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(30))`). When the watermark passes a window's end, the window fires and its state is freed.



Larger delay (60 s) → more complete windows, results 60 s later. Smaller (5 s) → faster, more late events.
With many partitions, the operator's watermark is the minimum of its inputs; an idle partition can stall it.

Figure 3. A 30-second watermark delay. The window fires when the watermark reaches its end; anything older that arrives afterwards is late.

Late events can be **dropped**, sent to a **side output** for separate handling, or accepted for an extra **allowed lateness** period during which the window re-fires with an updated result (so the sink must accept updates). The delay is a dial between completeness and latency; there is no setting that gives both.

5 State, checkpoints and exactly-once

Windows and counters are **state**, kept per key next to the computation (in Flink, on the heap or in an embedded RocksDB store, so it can exceed memory). The stream is partitioned by key (`keyBy(card)`), so all events for one card reach the same parallel instance and its local state.

- **Checkpoints:** Flink periodically injects **barriers** into the stream; when every operator has seen the barrier, it snapshots its state to durable storage (S3, HDFS) together with the Kafka offsets of that point. After a crash, the job restores the last checkpoint and re-reads from those offsets: state is **exactly-once** even though some events are processed twice.
- **End-to-end exactly-once** also needs the sink to cooperate: a transactional sink (Kafka transactions, two-phase commit tied to the checkpoint) or idempotent writes (upsert by key). Otherwise outputs between the checkpoint and the crash are emitted again.
- **Backpressure:** if a sink slows down, upstream operators slow too and Kafka buffers the backlog; watch consumer lag.

WORKED EXAMPLE: REAL-TIME FRAUD SCORING

Card payments flow into Kafka topic `payments`, partitioned by card number. A Flink job does `keyBy(card)` and, per card, keeps the last 5 minutes of payments in state (count, total, set of countries), with a 30 s bounded-out-of-orderness watermark to expire old entries by event time. Each new payment is evaluated *immediately* against that state rather than waiting for a window to close. Card 4321 makes its 5th payment within 2 minutes, in a 3rd country: the rule and a model score fire, and a `fraud_score` event is emitted in well under a second, so the authorisation service can block the next attempt. Checkpoints every 10 s let the job restart without losing the per-card history.

TRADE-OFFS

- Watermark delay: completeness vs latency.
- Checkpoint interval: frequent checkpoints mean less replay after a crash, but more I/O and, with transactional sinks, more latency.
- Event time is correct and replayable; processing time is simpler and lower latency but changes with network conditions.
- Large state (long windows, many keys) costs memory, disk and recovery time; set state TTLs.

COMMON MISTAKES

- Aggregating by processing time and then wondering why replays give different numbers.
- No plan for late events: silently dropping them makes totals drift from the batch numbers.
- An idle Kafka partition holding back the watermark, so no window ever fires.
- Assuming exactly-once state means exactly-once output without a transactional or idempotent sink.

6 Key terms

Event time: when the event happened at the source

Processing time: when the stream job handles it

Tumbling / sliding window: non-overlapping vs overlapping fixed windows

Session window: activity grouped until a gap of inactivity

Watermark: assertion that no older events are expected

Checkpoint: consistent snapshot of state + input offsets

7 Interview questions

Q1. What is a watermark? BASIC
The processor's estimate of how far event time has progressed (no older events expected), so it can close windows and emit results despite late, out-of-order data.

Q2. Tumbling versus sliding windows? BASIC
Tumbling windows are fixed and don't overlap; sliding windows have a fixed size but advance in smaller steps, so they overlap and each event counts in several.

Q3. Event time or processing time for a revenue-per-minute metric? BASIC
Event time: it assigns each sale to the minute it happened, so late arrivals and replays give the same answer.

Q4. Your per-minute counts are always 3% lower than the nightly batch. Why? SCENARIO
Late events arrive after the watermark and are dropped; increase the delay, use allowed lateness with updates, or reconcile with batch.

Q5. A Flink job crashes. How does it avoid double-counting? SCENARIO
It restores state from the last checkpoint and rewinds Kafka to the offsets stored with it; with a transactional or idempotent sink, outputs are also not duplicated.

Q6. No windows are firing although data flows. What could it be? TRICKY
The watermark is stuck: one input partition is idle or has a very old timestamp, and the operator takes the minimum; configure idleness detection or fix the source.

Q7. Why might fraud detection not use a windowed aggregate at all? TRICKY
A window emits only when it closes, adding latency; evaluating each event against per-key recent state answers immediately.

Next: SD-Q-008 · Lambda vs Kappa backendroadmap.com